

Assignment 2

1. Calculate the average waiting time and turnaround time for the given processes using the FCFS algorithm (5 points).

Processes	Arrival Time	Burst Time	TAT	WT
p1	6	6	11	5
p2	8	5	14	9
p3	0	3	3	0
p4	4	4	4	0
p5	5	3	6	3
Average			7.6	3.4

Process	P3	P4	P5	P1	P2
Time	0-3	4-8	8-11	11-17	17-22

-
2. Calculate the average waiting and turnaround time for round-robin with $q = 3$ (5 points).

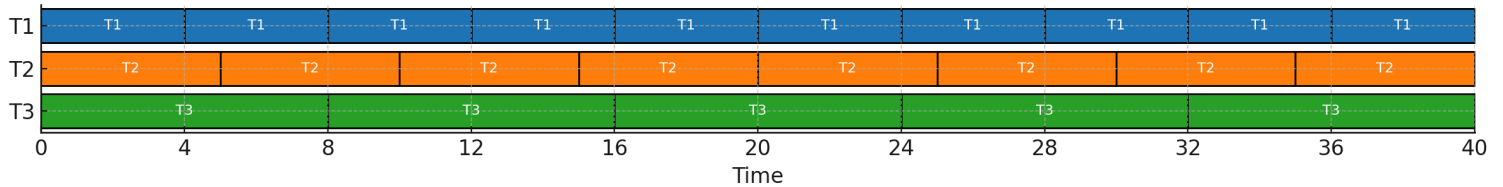
Process	Arrival Time	Burst Time	TAT	WT
P1	0	4	13	9
P2	1	3	5	2
P3	2	2	6	4
P4	3	3	8	5
P5	4	1	8	7
Average			8	5.4

Process	P1	P2	P3	P4	P5	P1
Time	0-3	3-6	6-8	8-11	11-12	12-13

3. You are given the following set of periodic tasks, apply rate monotonic algorithm and show the scheduling charts. (Consider context switching)(5 points).

Task	Capacity	Period
T1	1	4
T2	2	5
T3	2	8

Periodic Task Activation Chart



Task	T1	T2	T3	T1	T2	T3	T1	T3	T2	T1	T3	T2
Time	0-1	1-3	3-4	4-5	5-7	7-8	8-9	9-10	10-12	12-13	13-15	15-16

4. You are given the following set of tasks, where the deadline is not equal to the period (7 points).

Task	Capacity	Period	Deadline
T1	2	6	4
T2	1	10	7
T3	3	15	12

A. Show the scheduling chart for all the processes and check for any missed deadline. If a task is missing its deadline, indicate when it happens and explain why.

Task	T1	T2	T3	T1	IDLE	T2	IDLE	T1
Time	0-2	2-3	3-6	6-8	8-10	10-11	11-12	12-14

IDLE	T3	T1	T2	IDLE
14-15	15-18	18-20	20-21	21-24

- No task missed a deadline because the capacity's are quite short relative to their periods.

B. Explain the difference between deadline and period and how it influences the EDF scheduling decisions for this given set of processes.

- Tasks closer to hitting their deadline are higher priority than based on period or capacity.
 - If a task hits its deadline it just gives up and waits for the next period.
 - The deadline is shorter than period so it can't go over the period.
-

5. Explain the Critical Section Problem in your own words. Address the following points: (6 points)

- What challenges arise when multiple processes or threads share a common resource?
- Define mutual exclusion, progress, and bounded waiting, and explain why these properties are essential for any solution to the critical section problem.
- Provide an example of a race condition in a multi threaded program and explain how a lack of proper synchronisation can lead to incorrect behaviour.

The critical section problem is when you have multiple processes running (which is always), and 2 processes try to access the same piece of data at the same time. If both programs are trying to read the value and then update it, it could be updated by another process in between its reading and write operations (race condition).

Mutual exclusion just means only one process can enter the critical section at a time. With critical section meaning the piece of data that multiple processes might want to access.

The progress condition makes sure that if multiple processes are waiting, the one it lets in first doesn't depend on the result of any other interaction.

Bounded waiting basically means a process will be told to give up if it waits for the critical section for too long.

All together these prevent unpredictable data editing, deadlocks, and starvation.

6. Simulate a scenario where two processes attempt to enter their critical section at the same time. Explain step by step how the Test-and-Set lock works to prevent both from entering the critical section simultaneously. (2 points)
7. A variable X is stored in ram with the value of 0.
8. Process 1 wants to increment X by 2.
9. Process 2 wants to increment X by 1.

10. P1 and P2 both try to access X at the same time.
11. P1 happens to start first, so it locks the critical section.
12. P2 starts and sees that X is locked.
13. P1 reads X as 0.
14. X gets incremented by 2.
15. P1 unlocks X.
16. P2 sees that X is unlocked and increments X to 3.

Without locks the outcome could have been 1, 2, or 3 with no way of knowing.